

Stochastic Modelling

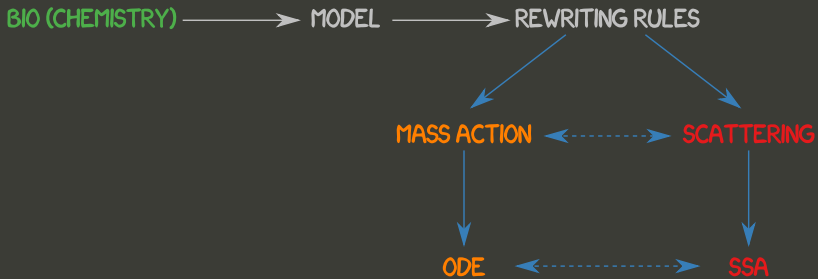
Dario Pescini¹

Università degli Studi di Milano-Bicocca

Dipartimento di Statistica e Metodi Quantitativi

dario.pescini@unimib.it

Outline



Scattering perspective

In this framework molecules are “hard-spheres” with radius r_i , position $\vec{x}_i$ and speed $\vec{v}_i$

Scattering perspective

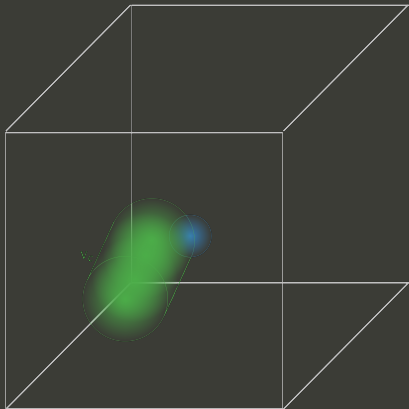
The rate of collisions $S_1 S_2$ will be proportional to the relative speed $\vec{v}_{1,2}$ of the two molecules.

Scattering perspective

In the next time interval δt molecule 1 will sweep a "collision" volume

$$\delta V_c = \pi(r_1 + r_2)^2 |\vec{v}_{1,2}| \delta t$$

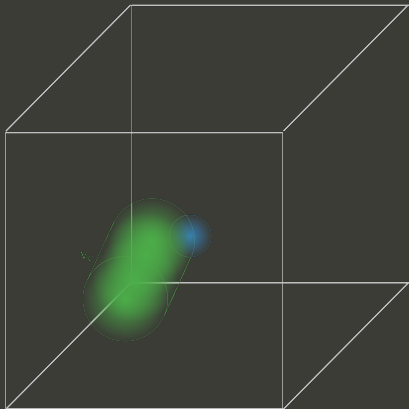
Scattering perspective



The probability of collision $P_{1,2}$ is related to the probability for molecule 2 to be in δV_c :

$$P_{1,2} = \frac{\delta V_c}{V}$$

Scattering perspective



Having n_1 and n_2 molecules in V the probability to have a reaction among them becomes:

$$\begin{aligned}
 P_{1,2} &= n_1 n_2 \frac{\delta V_c}{V} = \\
 &= n_1 n_2 \frac{\pi(r_1 + r_2)^2 |\vec{v}_{1,2}| \delta t}{V}
 \end{aligned}$$

Scattering perspective

In:

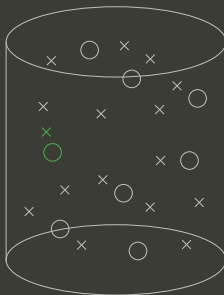
$$P_{1,2} = n_1 n_2 \frac{\pi(r_1 + r_2)^2 |\vec{v}_{1,2}|}{V} \delta t$$

we can recognize two distinct contributions:

- **combinatorics** $n_1 n_2$
- **physics**
 - molecules shape $\pi(r_1 + r_2)^2$
 - system energy $|\vec{v}_{1,2}|$

Increasing accuracy of the combinatorics contribution

Let's start counting the collisions between **couples of molecules**:

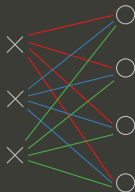


given species X and O there are two possible kinds of couples:

- $X + O \longrightarrow C$ heterologous
- $X + X \longrightarrow C$ homologous

Counting the couples

- Heterologous:



$$n_x \cdot n_o = 3 \cdot 4 = 12$$

- Homologous:



$$\frac{n_o(n_o - 1)}{2} = \frac{4 \cdot 3}{2} = 6$$

Three molecules... and more

Heterologous:

$$O \ X \ X = O(X \ X) \quad \mapsto \quad n_o \cdot \frac{n_x(n_x - 1)}{2} \quad \mapsto \quad n_o \binom{n_x}{2}$$

Homologous:

$$X \ X \ X \quad \mapsto \quad \frac{n_x \cdot (n_x - 1)(n_x - 2)}{3!} \quad \mapsto \quad \binom{n_x}{3}$$

Three molecules... and more

Heterologous:

$$O\ X\ X = O(X\ X) \quad \longmapsto \quad n_o \cdot \frac{n_x(n_x - 1)}{2} \quad \longmapsto \quad n_o \binom{n_x}{2}$$

Homologous:

$$X\ X\ X \quad \longmapsto \quad \frac{n_x \cdot (n_x - 1)(n_x - 2)}{3!} \quad \longmapsto \quad \binom{n_x}{3}$$

$$\alpha_1 x_1 + \alpha_2 x_2 + \cdots + \alpha_N x_N \quad \longmapsto \quad \prod_{h=1}^N \binom{x_i}{\alpha_i}$$

From the scattering perspective to an algorithmic approach: SSA

Stochastic Simulation Algorithm (SSA - Gillespie '76)

General Problem:

- V = fixed volume
- $\{S_i\}_{i=1,\dots,N}$ = set of N chemical species in V
- $\{X_i\}_{i=1,\dots,N}$ = current number of molecules in V
- $\{R_\mu\}_{\mu=1,\dots,M}$ = set of M chemical reactions
- $\{c_\mu\}_{\mu=1,\dots,M}$ = set of M reactions parameters

How does it evolve?

Fundamental Hypothesis:

average probability that a particular combination R_μ
 $c_\mu \delta t$ = of reactant molecules will react accordingly in the next time
 interval δt .

SSA: How does it work?

∀ dynamical step the algorithm answers 2 questions:

- **when** the next reaction will occur? τ
- **which one** the next reaction will be? μ

this is done exploiting the Fundamental Hypothesis to get

probability that the next reaction in V will occur in the
 $P(\tau, \mu) \delta\tau =$ differential time interval $(t + \tau, t + \tau + d\tau)$ and will be
 R_μ

$P(\mu, \tau) \propto a_\mu = c_\mu h_\mu$ **propensity function**

c_μ summarizes **chemical** and **physical** properties

$$h_\mu = \prod_{i=1}^N \binom{X_i}{\alpha_i} \quad \text{combinatorics}$$

$$a_0 = \sum_{\mu=1}^M a_\mu \quad \text{normalization}$$

∀ dynamics step

In a rigorous way it can be shown that:

$$P(\tau, \mu) = P_1(\tau) P_2(\mu)$$

where

$$P_1(\tau) = a_0 e^{-a_0 \tau}$$

$$P_2(\mu) = \frac{a_\mu}{a_0}$$

∀ dynamics step

so that the **answer** to the 2 questions is the pair (τ, μ)

$$\tau = \frac{1}{a_0} \ln \frac{1}{r_1}$$

$$\sum_{\nu=1}^{\mu-1} a_{\nu} < r_2 a_0 < \sum_{\nu=1}^{\mu} a_{\nu}$$

τ : RNG inversion Method

We wish to generate x according to a **probability density function** $P(x)$:

$$F(x) = \int_{-\infty}^x P(x') dx' \quad F(x) \text{ probability distribution function}$$

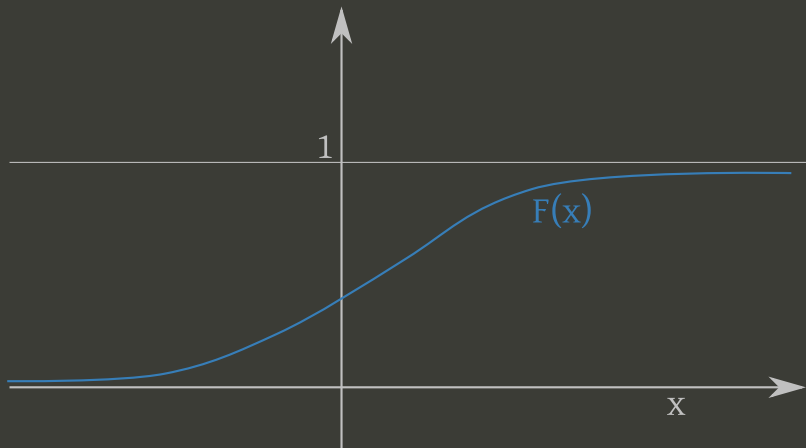
Let

- $F(-\infty) = 0$
- $F(\infty) = 1$
- $F(x)$ monotone, invertible
- $r \in \text{UR}[0, 1]$

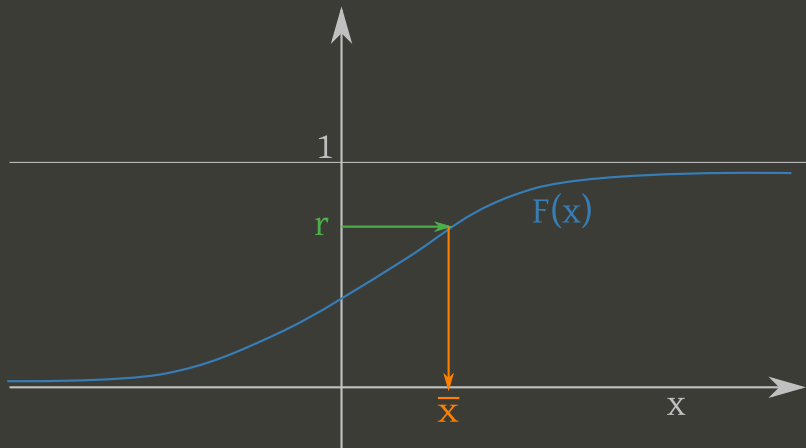
if $r = F(x)$ then $x = F^{-1}(r)$

F^{-1} is the **inverse distribution** corresponding to the given density function

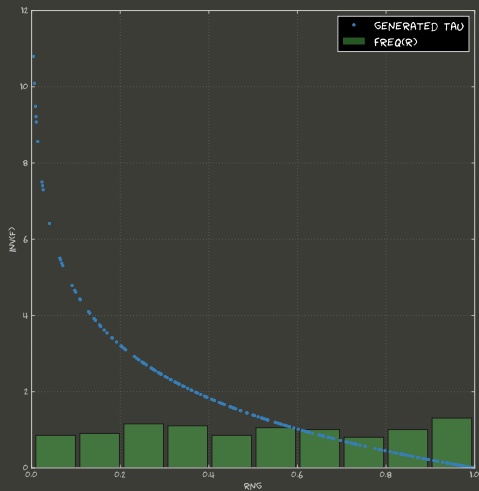
RNG: inversion Method



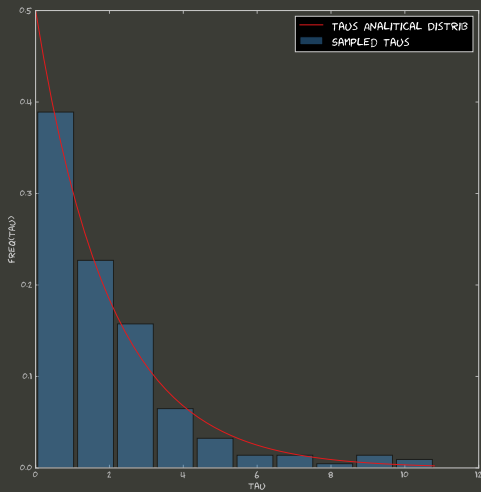
RNG: inversion Method



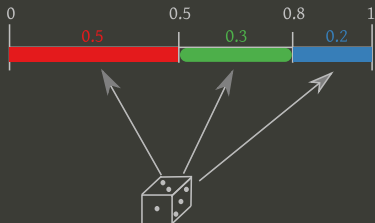
RNG: inversion Method

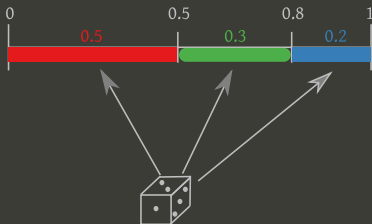


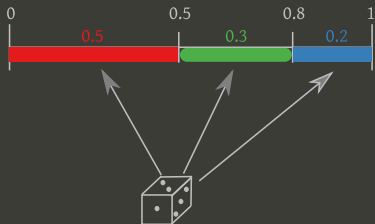
RNG: inversion Method



Rule Tossing: μ



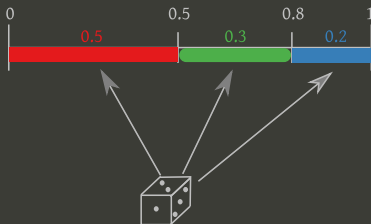
Rule Tossing: μ 

Rule Tossing: μ 

SSA: The Algorithm

for each step of the dynamics:

- compute the **probability distribution**
- **toss** r_1



- **toss** r_2 and compute $\tau = \frac{1}{a_0} \ln \frac{1}{r_2}$
- then **apply the tossed rule** to modify the number of molecules involved by that rule
- then **update** the evolution time $t \leftarrow t + \tau$

SSA: an implementation (Python)

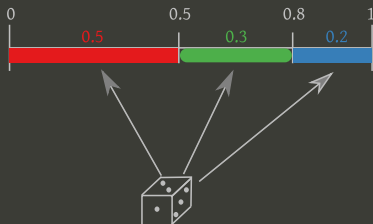
c_μ summarizes chemical and physical properties

$$h_\mu = \prod_i^N \binom{X_i}{\alpha_i} \quad \text{combinatorics}$$

$$a_0 = \sum_{\mu}^M a_\mu \quad \text{normalization}$$

```
def propensities(props, xIn, cIn, leftSideIn):
    leftDim = leftSideIn.shape
    for row in xrange(leftDim[0]):
        props[row] = cIn[row]
        for col in xrange(leftDim[1]):
            props[row] *= binCoeff(xIn[col], leftSideIn[row,
col])
```

SSA: an implementation



```
def tossRule(propensities, aZero):  
    aProbs = propensities / aZero  
    rulesSep = aProbs.cumsum()  
    mu = np.where(np.random.random() < rulesSep)[0][0]  
    return mu
```

SSA: an implementation

then **apply the tossed rule** to modify the number of molecules involved by that rule

```
def applyRule(deltaMatrix, iRule, aX):  
    aX += deltaMatrix[iRule][:]  
    return mu
```

SSA: an implementation (Python)

```
iRulesLines, iSpecsLines, iFeedsLines =  
    mP.findRulesAndSpecs(fileModelName)  
aRules, aConstants = mP.rulesAndConstants(fileModelName,  
    iRulesLines)  
dictAlphabet = mP.rulesToAlphabet(aRules)  
leftSide, rightSide, delta = mP.rulesToMatrices(aRules,  
    dictAlphabet)  
  
X = mP.speciesInit(fileModelName, iSpecsLines,  
    dictAlphabet).astype(int)  
dictFeedSpecs = mP.speciesInFeed(fileModelName, iFeedsLines,  
    dictAlphabet, X)  
  
aPropensities = np.zeros(len(aRules), dtype=float)  
t = 0.0  
while (t < tMax):  
    propensities(aPropensities, X, aConstants, leftSide)  
    a0 = aPropensities.sum()  
    deltaT = tossTau(a0)  
    rule = tossRule(aPropensities, a0)  
    t += deltaT  
    applyRule(delta, rule, X)  
    feedStep(X, dictFeedSpecs)
```

the BZ Reaction and the Brusselator: the Dynamics

